



AGH

AGH UNIVERSITY OF SCIENCE
AND TECHNOLOGY

PROGRAMOWANIE GRAFIKI 3D

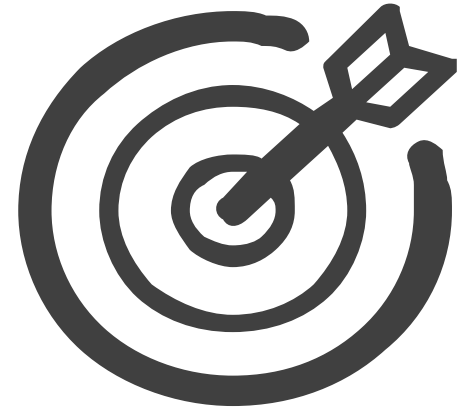
Standard X3D

dr hab. inż. Marcin Hojny, prof. AGH

Faculty of Metals Engineering and Industrial Computer Science
Department of Applied Computer Science and Modelling

CEL ZAJĘĆ?

-  CZUJNIKI DOTYKOWE;
-  CZUJNIKI OBROTU I PRZESUNIĘCIA;
-  CZUJNIKI ŚRODOWISKOWE;
-  HORYZONT, PANORAMA;
-  MGŁA;
-  TWORZENIE ZŁOŻONYCH OBIEKTÓW;
-  DODATKOWE WĘZŁY I POLECENIA (GROUP, WHICH, PROTO);



CZUJNIKI DOTYKOWE – WĘZEŁ ANCHOR

Wykorzystując węzeł **Anchor** możemy stworzyć link między światem X3D a innym miejscem w sieci. Może on także odsyłać do miejsc zdefiniowanych w tym samym świecie za pomocą węzła **Viewpoint**. Interfejs węzła **Anchor** jest następujący:

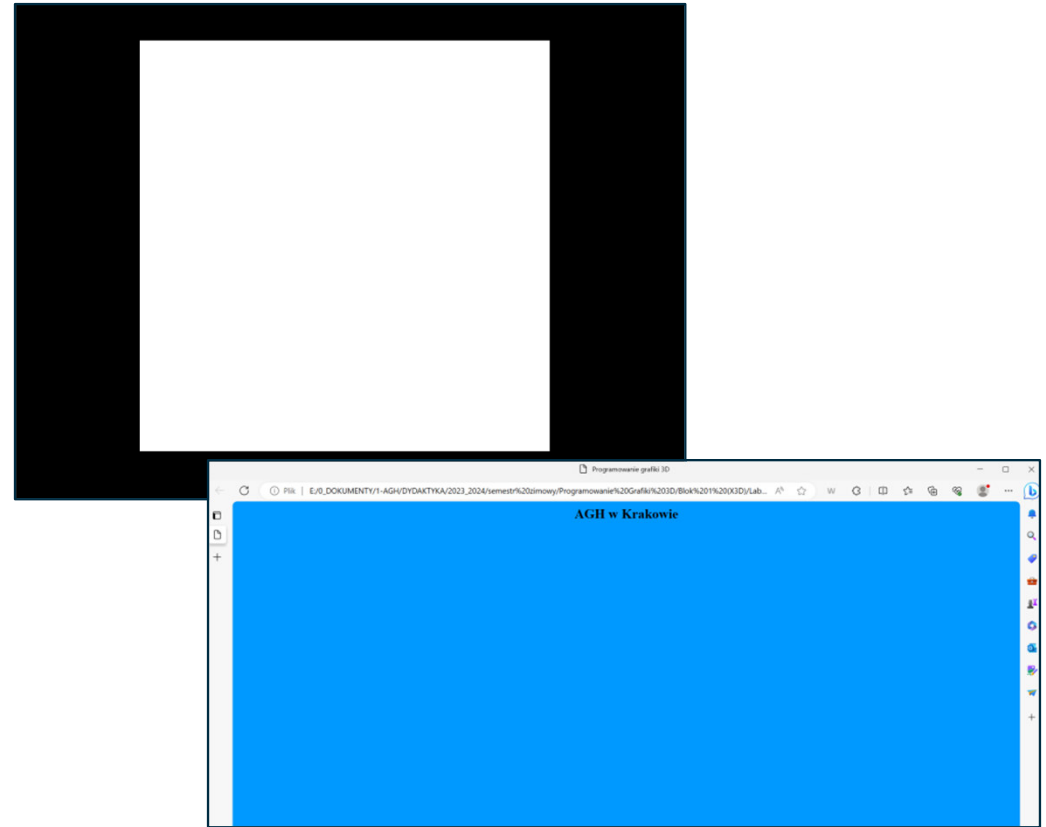
```
Anchor {  
  eventIn    MFNode  addChildren  
  eventIn    MFNode  removeChildren  
  exposedField MFNode  children      []  
  exposedField SFString description  ""  
  exposedField MFString parameter    []  
  exposedField MFString url          []  
  field      SFVec3f  bboxCenter      0 0 0  
  field      SFVec3f  bboxSize        -1 -1 -1  
}
```

CZUJNIKI DOTYKOWE – WĘZEL ANCHOR

PRZYKŁAD 1: dowolny plik który obsługuje nasza przeglądarka

#X3D V3.3 utf8

```
Anchor {  
  url "index.html"  
  children [  
    Shape { geometry Box {} }  
  ]  
}
```



CZUJNIKI DOTYKOWE – WĘZEL ANCHOR

PRZYKŁAD 2: link do innego miejsca Viewpoint tego samego świata

```
#X3D V3.3 utf8
```

```
Anchor {  
  url "#Viewpoint 2"  
  children [  
    Shape { geometry Box {} }  
  ]  
}
```

Powyższa konstrukcja przeniesie użytkownika obecnego świata do miejsca o nazwie „**Viewpoint 2**” zdefiniowanego w węźle **Viewpoint**.

CZUJNIKI DOTYKOWE – WĘZEL ANCHOR

PRZYKŁAD 3: link do innego miejsca Viewpoint w innym świecie

```
#X3D V3.3 utf8
```

```
Anchor {  
  url „inny_swiat#Viewpoint 3”  
  children [  
    Shape { geometry Box {} }  
  ]  
}
```

Powyższa konstrukcja przeniesie użytkownika obecnego świata do świata o nazwie **inny_swiat** i miejsca o nazwie „**Viewpoint 3**” zdefiniowanego w węźle **Viewpoint**.

CZUJNIKI DOTYKOWE

– WĘZEŁ TOUCHSENSOR

```
TouchSensor {  
  exposedField SFBool enabled TRUE  
  eventOut    SFVec3f hitNormal_changed  
  eventOut    SFVec3f hitPoint_changed  
  eventOut    SFVec2f hitTexCoord_changed  
  eventOut    SFBool  isActive  
  eventOut    SFBool  isOver  
  eventOut    SFTime  touchTime  
}
```

CZUJNIKI DOTYKOWE

– WĘZEŁ TOUCHSENSOR

W przypadku gdy pole **enabled** będzie miało wartość **TRUE** oraz „klikniemy” na aktywny obiekt lub najedziemy na niego kursorem myszy wysyłane zostają pozostałe zdarzenia:

- Podczas przesunięcia kursora z miejsca nie aktywnego w aktywne zdarzenie **isOver** wysyła wartość **TRUE**. W sytuacji odwrotnej zdarzenie **isOver** wysyła wartość **FALSE**.
- W przypadku gdy zdarzenie **isOver** wyśle wartość **TRUE** zostaną wysyłane również zdarzenia **hitPoint_changed**, **hitNormal_changed** i **hitTexCoord_changed**. Wartość wysyłana wraz ze zdarzeniem **hitPoint_changed** zawiera definicję punktu (x, y, z) na powierzchni obiektu objętego działaniem węzła **TouchSensor**. Zdarzenie **hitNormal_changed** wysyła informację o wektorze prostopadłym do aktywnej powierzchni stykający się z powierzchnią w punkcie wyznaczonym przez zdarzenie **hitPoint_changed**. Zdarzenie **hitTexCoord_changed** zawiera informację o współrzędnych tekstury nałożonej na obiekt aktywny od punktu wyznaczonego przez zdarzenie **hitPoint_changed**.

CZUJNIKI DOTYKOWE

– WĘZEŁ TOUCHSENSOR

- W przypadku gdy zdarzenie **isOver** ma wartość **TRUE** „kliknięcie” klawiszem myszy spowoduje wysłanie zdarzenia **isActive** o wartości **TRUE**. *Jeżeli przytrzymamy klawisz myszki zdarzenie **isActive** spowoduje, że na czas przytrzymania klawisza myszka zostanie wyłączona z innych zadań aż do czasu gdy klawisz zostanie puszczony.*
- Zdarzenie **touchTime** jest wysyłane w momencie gdy trzy poniższe warunki zostaną spełnione:
 - kursor myszki był skierowany na obiekt aktywny a klawisz myszki wciśnięty w momencie gdy zdarzenie **isActive** było **TRUE**,
 - kursor myszki w danej chwili jest skierowany na aktywny obiekt, czyli wartość zdarzenie **isOver** ma wartość **TRUE**,
 - klawisz myszki zostanie puszczony - zdarzenie **isActive** przyjmuje wartość **FALSE**.

CZUJNIKI OBROTU I PRZESUNIĘCIA

Do grupy czujników obrotu i przesunięcia zaliczamy następujące węzły:

- **PlaneSensor,**
- **CylinderSensor,**
- **SphereSensor.**

Działanie powyższych czujników polega na tym, że w momencie przytrzymania klawisza myszy na aktywnym obiekcie, można go przesuwąć lub obracać (w zależności od rodzaju użytego węzła czujnikowego).

CZUJNIKI OBROTU I PRZESUNIĘCIA

Wspólne pola dla węzłów **PlaneSensor**, **CylinderSensor** i **SphereSensor**:

- Pole **offset** i **autoOffset**. Pola te odpowiadają za zapamiętanie stanu ostatniego przesunięcia lub obrotu obiektu. W momencie gdy **autoOffset** ma ustawioną wartość **TRUE** to miejsce, w którym obiekt został zostawiony podczas ostatniego korzystania z węzła czujnikowego zostaje zapamiętane i przy ponownym użyciu tego węzła, ruch rozpoczyna się właśnie od tego miejsca.

CZUJNIKI OBROTU I PRZESUNIĘCIA

- Pole **maxPosition** i **minPosition** wyznaczają granice wychylenia obiektu. W momencie gdy pole **autoOffset** ma ustawioną wartość **FALSE** można zdefiniować własny punkt, z którego zacznie się przesuwanie obiektu przy uaktywnieniu tych węzłów czujnikowych. Punkt ten definiuje się w polu **offset**.
- Zdarzenie **trackPoint_changed**, które zostaje wysłane w chwili uaktywnienia węzła. Zdarzenie to „śledzi” pozycję kursora, w związku z czym jeżeli połączymy zdarzenie **trackPoint_changed** ze zdarzeniem **set_translation** węzła **Transform**, w którym znajduje się definicja obiektu, to obiekt będzie się przesuwał w ślad za kursorem myszki.

CZUJNIKI OBROTU I PRZESUNIĘCIA

```
PlaneSensor {  
  exposedField SFBool autoOffset      TRUE  
  exposedField SFBool enabled          TRUE  
  exposedField SFVec2f maxPosition     -1 -1  
  exposedField SFVec2f minPosition     0 0  
  exposedField SFVec3f offset          0 0 0  
  eventOut    SFBool isActive  
  eventOut    SFVec3f trackPoint_changed  
  eventOut    SFVec3f translation_changed  
}
```

Węzeł **PlaneSensor** umożliwia przesuwanie obiektów względem osi X i Y oraz umożliwia ograniczenie przesunięcia obiektu. W polach **maxPosition** i **minPosition** określa się granice, w których obiekt może być przemieszczany. Pierwsza wartość w tych polach dotyczy osi X a druga osi Y. Pozostawienie wartości domyślnych spowoduje, że można przemieszczać aktywny obiekt bez ograniczeń.

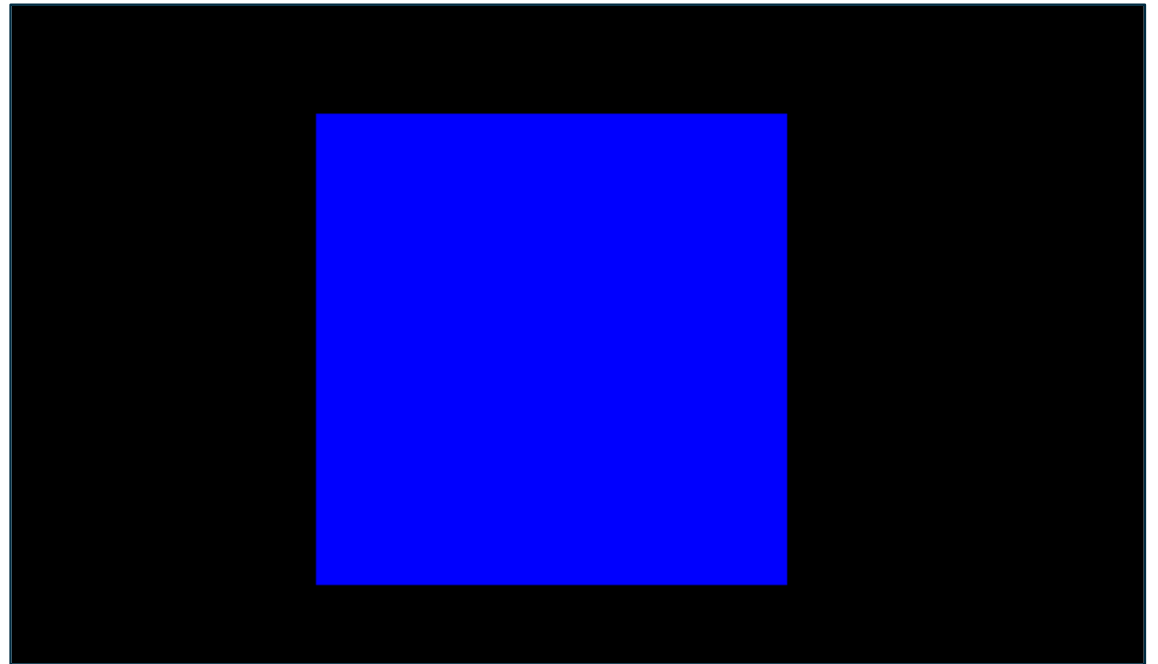
CZUJNIKI OBROTU I PRZESUNIĘCIA

#X3D V3.3 utf8

```
DEF Obiekt Transform {  
  children [  
    Shape {  
      appearance Appearance { material Material {  
        diffuseColor 0 0 1 }  
      }  
      geometry Box {size 3 3 0.1} }  
  ]  
}
```

```
DEF Czujnik PlaneSensor {  
  autoOffset FALSE  
  offset 0 0 0  
  minPosition -1 -1  
  maxPosition 2 2 }
```

```
ROUTE Czujnik.translation_changed TO Obiekt.set_translation
```



CZUJNIKI OBROTU I PRZESUNIĘCIA

```
CylinderSensor {  
  exposedField SFBool    autoOffset TRUE  
  exposedField SFFloat   diskAngle 0.262  
  exposedField SFBool    enabled   TRUE  
  exposedField SFFloat   maxAngle  -1  
  exposedField SFFloat   minAngle  0  
  exposedField SFFloat   offset    0  
  eventOut   SFBool    isActive  
  eventOut   SFRotation rotation_changed  
  eventOut   SFVec3f   trackPoint_changed  
}
```

Działanie węzła **CylinderSensor** jest podobnie do działania węzła **PlaneSensor**. Różnica polega na tym, że aktywny obiekt nie jest przesuwany a obracany tylko względem osi Y.

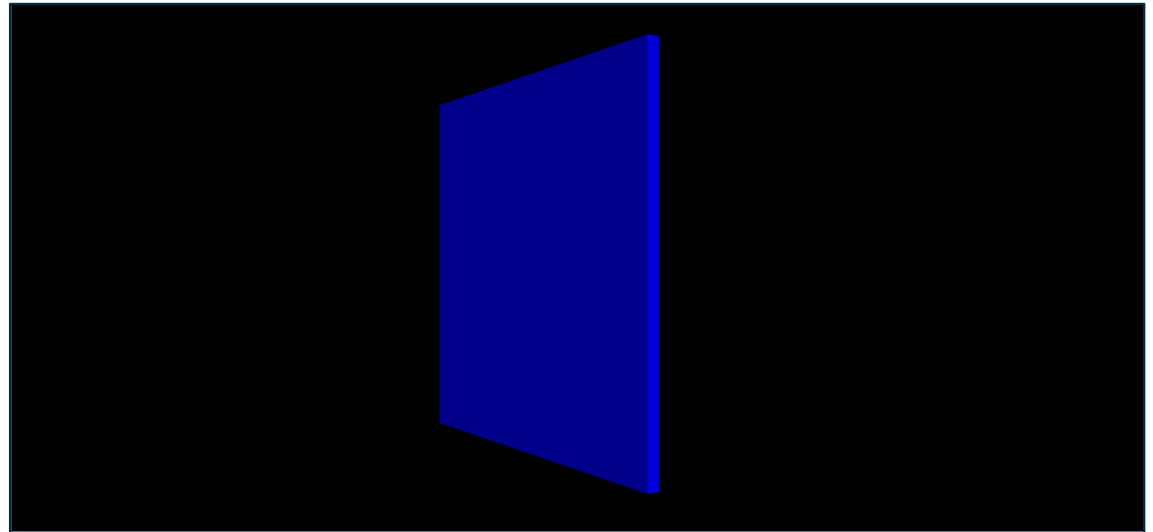
CZUJNIKI OBROTU I PRZESUNIĘCIA

#X3D V3.3 utf8

```
DEF Obiekt Transform {  
  children [  
    Shape {  
      appearance Appearance {  
        material Material {  
          diffuseColor 0 0 1 }  
        }  
      geometry Box {size 3 3 0.1}  
    }  
  ]  
}
```

```
DEF Czujnik CylinderSensor {  
  autoOffset TRUE  
  minAngle -1  
  maxAngle 1 }
```

```
ROUTE Czujnik.rotation_changed TO Obiekt.set_rotation
```



CZUJNIKI OBROTU I PRZESUNIĘCIA

```
SphereSensor {  
  exposedField SFBool    autoOffset    TRUE  
  exposedField SFBool    enabled      TRUE  
  exposedField SFRotation offset        0 1 0 0  
  eventOut    SFBool    isActive  
  eventOut    SFRotation rotation_changed  
  eventOut    SFVec3f   trackPoint_changed  
}
```

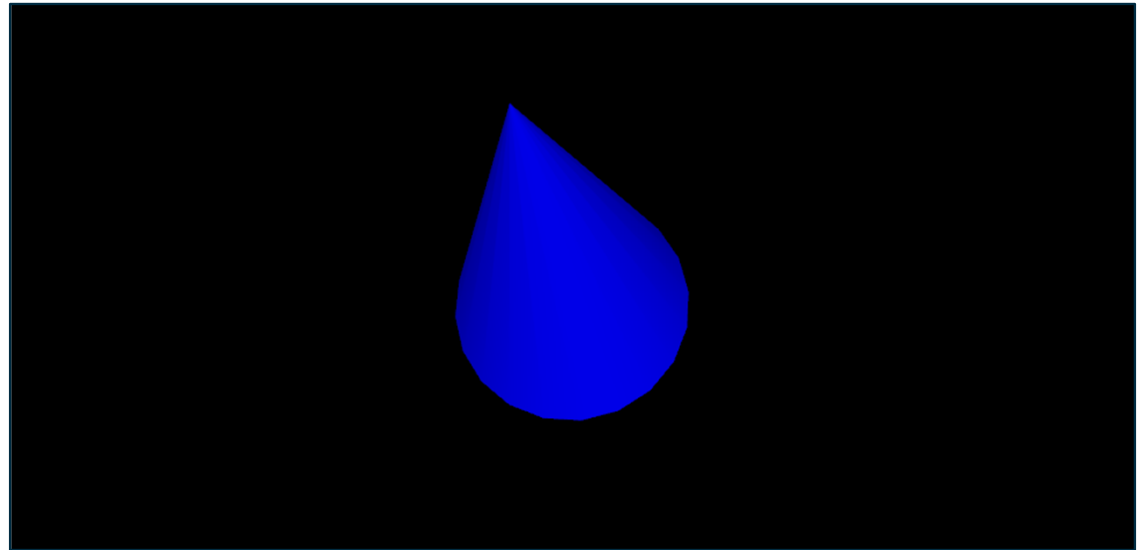
Węzeł **SphereSensor** działa analogicznie jak **CylinderSensor**. Wykorzystanie węzła **SphereSensor** umożliwia obrót danego obiektu we wszystkich kierunkach.

CZUJNIKI OBROTU I PRZESUNIĘCIA

#X3D V3.3 utf8

```
DEF Obiekt Transform {  
  children [  
    Shape {  
      appearance Appearance {  
        material Material {  
          diffuseColor 0 0 1 }  
        }  
      geometry Cone { height 4  
        }  
    }  
  ]  
}  
  
DEF Czujnik SphereSensor {  
  enabled TRUE  
  autoOffset TRUE }
```

```
ROUTE Czujnik.rotation_changed TO Obiekt.set_rotation
```



CZUJNIKI ŚRODOWISKOWE

Czujniki środowiskowe są uruchamiane przez użytkownika nieświadomie, tzn. do ich uaktywnienia nie potrzebne jest świadomie działanie użytkownika.

Do grupy czujników środowiskowych zaliczamy węzły:

- **Collision,**
- **ProximitySensor,**
- **VisibilitySensor.**

CZUJNIKI ŚRODOWISKOWE

```
Collision {
    eventIn    MFNode  addChildren
    eventIn    MFNode  removeChildren
    exposedField MFNode  children    []
    exposedField SFBool  collide      TRUE
    field      SFVec3f  bboxCenter    0 0 0
    field      SFVec3f  bboxSize      -1 -1 -1
    field      SFNode   proxy         NULL
    eventOut   SFTIME   collideTime
}
```

Collision jest węzłem grupującym określającym właściwości kolizji dla obiektów zdefiniowanych w polu **children**. Pole **collide** odpowiada za **włączenie lub wyłączenie** wykrywania kolizji z obiektami zdefiniowanymi w węźle **Collision**. Pole **proxy** definiuje obiekt który jest niewidoczny, o który się zatrzymamy chcąc dotrzeć do danego obiektu. Jeżeli w polu **children** nie znajdzie się definicja żadnego obiektu, pole **collide** będzie miało wartość **TRUE**, a w polu **proxy** zostanie zdefiniowany obiekt to nic nie zostanie wyświetlone na ekranie, ale w ten właśnie sposób można zdefiniować kolizje z niewidzialnym obiektem. Przy zderzeniu z obiektami utworzonymi w polu **proxy** węzeł również wysyła odpowiednie zdarzenia. Podczas kolizji, węzeł **Collision** wysyła zdarzenie **collideTime**, które można powiązać z innymi zdarzeniami wywołując jakiś efekt, który ma kolizji towarzyszyć. Jednak aby zdarzenie **collideTime** mogło zostać wysłane pole **collide** musi mieć wartość **TRUE**.

CZUJNIKI ŚRODOWISKOWE

#X3D V3.3 utf8

```
NavigationInfo {  
    speed 10.0  
}
```

```
Transform {  
    children [  
        Collision {  
            collide TRUE  
            children [  
                Shape {  
                    appearance Appearance { material Material {  
                        diffuseColor 1 0.5 0 }  
                    }  
                    geometry Box {size 10 10 0.1} } ] }  
            ]  
        }  
    }
```

```
Transform {  
    translation 0, 0, -25  
    children [  
        Shape {  
            appearance Appearance { material Material {  
                diffuseColor 0 1 0 }  
            }  
            geometry Box {size 1 1 1} }  
    ]  
}
```

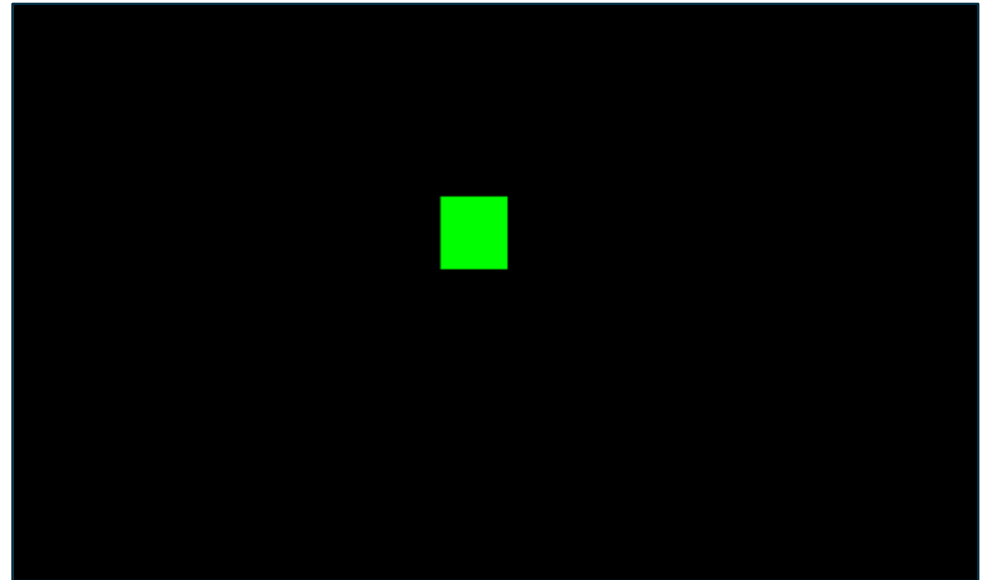
CZUJNIKI ŚRODOWISKOWE

PRZYKŁAD: PRZEJŚCIE PRZEZ „ŚCIANĘ” .

collide TRUE



collide FALSE



CZUJNIKI ŚRODOWISKOWE

```
ProximitySensor {  
  exposedField SFVec3f   center    0 0 0  
  exposedField SFVec3f   size      0 0 0  
  exposedField SFBool    enabled   TRUE  
  eventOut    SFBool     isActive  
  eventOut    SFVec3f    position_changed  
  eventOut    SFRotation  orientation_changed  
  eventOut    SFTIME      enterTime  
  eventOut    SFTIME      exitTime  
}
```

CZUJNIKI ŚRODOWISKOWE

Wykorzystując węzeł **ProximitySensor** można zdefiniować obszar aktywny na wkroczenie użytkownika. W momencie wkroczenia użytkownika w aktywny obszar węzeł wysyła grupę zdarzeń. Aktywny obszar jest określony niewidocznym sześcianem, którego środek określany jest w polu **center**, a rozmiar w polu **size**.

Kiedy użytkownik wkracza w zdefiniowaną przestrzeń wysyłane jest zdarzenie **isActive** o wartości **TRUE** oraz zdarzenie **enterTime**, natomiast kiedy użytkownik opuszcza daną przestrzeń, wysyłane jest zdarzenie **isActive** o wartości **FALSE** oraz zdarzenie **exitTime**.

Zdarzenia **position_changed** i **orientation_changed** "śledzą" ruch uczestnika świata i wysyłają zdarzenia, które mogą być odczytywane przez inne węzły. Jeżeli węzeł **ProximitySensor** ponownie przywołamy w innym miejscu używając np.: mechanizmu **DEF/USE** to wtedy aktywna staje się przestrzeń, którą stanowi suma "przestrzeni" ją wyznaczających.

CZUJNIKI ŚRODOWISKOWE

PRZYKŁAD: AKTYWOWANIE RUCHU KULI.

#X3D V3.3 utf8

```
Transform {  
    translation 0 0 0  
    children [  
        Collision {  
            collide FALSE  
        }  
        children [  
            DEF Prox ProximitySensor {size 2 2 2}  
                Shape { appearance Appearance {  
material Material { transparency 0.6 }  
                }  
            geometry Box {size 2 2 2}  
            }  
        ]  
    ]  
}
```

```
DEF Obiekt Transform {  
    translation -10 0 -20  
    children [  
        Shape { appearance Appearance {  
Material Material { diffuseColor 0 1 0 }  
        }  
        geometry Sphere {}  
    ]  
}  
  
DEF Czas TimeSensor {cycleInterval 5}  
  
DEF Ruch PositionInterpolator {  
    key [ 0, 0.25, 0.5, 0.75 , 1 ]  
    keyValue [ -5 0 -20, 0 5 -20, 5 0 -20, 0 -5 -20, -5 0 -20 ]  
}  
  
ROUTE Czas.fraction_changed TO Ruch.set_fraction  
ROUTE Ruch.value_changed TO Obiekt.set_translation  
  
ROUTE Prox.enterTime TO Czas.set_startTime  
ROUTE Prox.isActive TO Czas.set_loop
```



CZUJNIKI ŚRODOWISKOWE

```
VisibilitySensor {  
  exposedField SFVec3f center  0 0 0  
  exposedField SFBool  enabled TRUE  
  exposedField SFVec3f size    0 0 0  
  eventOut    SFTIME enterTime  
  eventOut    SFTIME exitTime  
  eventOut    SFBool  isActive  
}
```

Węzeł **VisibilitySensor** "uwrażliwia" zdefiniowany obszar na spojrzenie użytkownika. Budowa węzła **VisibilitySensor** jest analogiczna do budowy węzła **ProximitySensor**. W obu węzłach identycznie działają zdarzenia oraz pola **center**, **size** i **enabled**.

CZUJNIKI ŚRODOWISKOWE

PRZYKŁAD: ZMIANA KOLORÓW CYLINDRÓW.

#X3D V3.3 utf8

```
Transform {  
    children [  
        DEF Obiekt Shape {  
            appearance Appearance {  
                material DEF Kolor Material  
                { diffuseColor 0 1 1 }  
            }  
            geometry Cylinder { }  
        }  
    ]  
}
```

```
DEF Obszar VisibilitySensor {  
    center 0 0 10  
    size 1 1 1  
}
```

```
DEF Czas TimeSensor { }
```

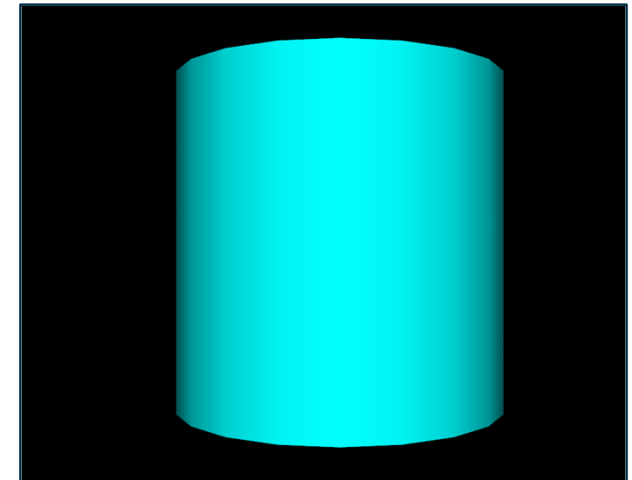
```
DEF Czas2 TimeSensor { }
```

```
DEF Animacja ColorInterpolator {  
    key [0, 1]  
    keyValue [1 0 0, 1 1 0 ]  
}
```

```
DEF Animacja2 ColorInterpolator {  
    key [0, 1]  
    keyValue [1 1 0, 1 0 0 ]  
}
```

```
ROUTE Obszar.enterTime TO Czas.set_startTime  
ROUTE Czas.fraction_changed TO Animacja.set_fraction  
ROUTE Animacja.value_changed TO Kolor.set_diffuseColor
```

```
ROUTE Obszar.exitTime TO Czas2.set_startTime  
ROUTE Czas2.fraction_changed TO Animacja2.set_fraction  
ROUTE Animacja2.value_changed TO Kolor.set_diffuseColor
```



HORYZONT, PANORAMA I MGŁA – WĘZŁY BACKGROUND I FOG

Wykorzystując węzeł **Background** można dokładnie określić kolor "nieba" oraz "ziemi" wirtualnego świata.

```
Background {
  eventIn    SFBool  set_bind
  exposedField MFFloat groundAngle []
  exposedField MFColor groundColor []
  exposedField MFString backUrl    []
  exposedField MFString bottomUrl  []
  exposedField MFString frontUrl   []
  exposedField MFString leftUrl    []
  exposedField MFString rightUrl   []
  exposedField MFString topUrl     []
  exposedField MFFloat skyAngle    []
  exposedField MFColor skyColor    0 0 0
  eventOut    SFBool  isBound
}
```

HORYZONT, PANORAMA I MGŁA – WĘZŁY BACKGROUND I FOG

Kolor "nieba" i "ziemi" określa się korzystając z pól **groundAngle**, **groundColor** oraz **skyAngle** i **skyColor**.

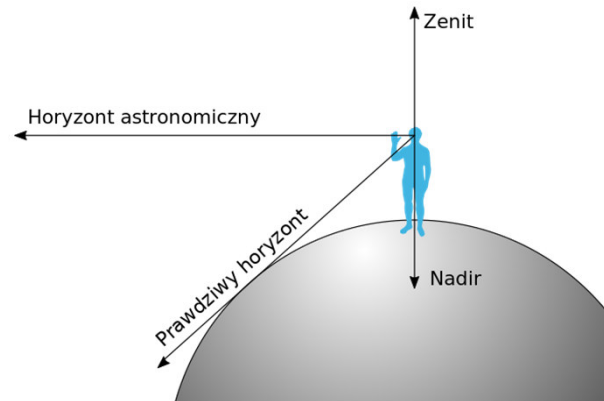
W polu **skyAngle** wyznacza się kąt tworzący koncentryczny pas wokół zenitu, a w polu **skyColor** określa się barwę, jaka ma się w tym pasie znaleźć (barwę określamy w standardzie RGB). Takich pasów możemy utworzyć bardzo wiele, ale zawsze w polu **skyColor** musi znajdować się jedna definicja koloru więcej, niż jest zdefiniowanych kątów. Pierwsza wartość w polu **skyColor** przyjmowana jest za kolor dla zenitu.

Wygląd „ziemi” tworzy się w sposób analogiczny, tylko że kąty w polu **groundAngle** liczone są od **nadiru**.

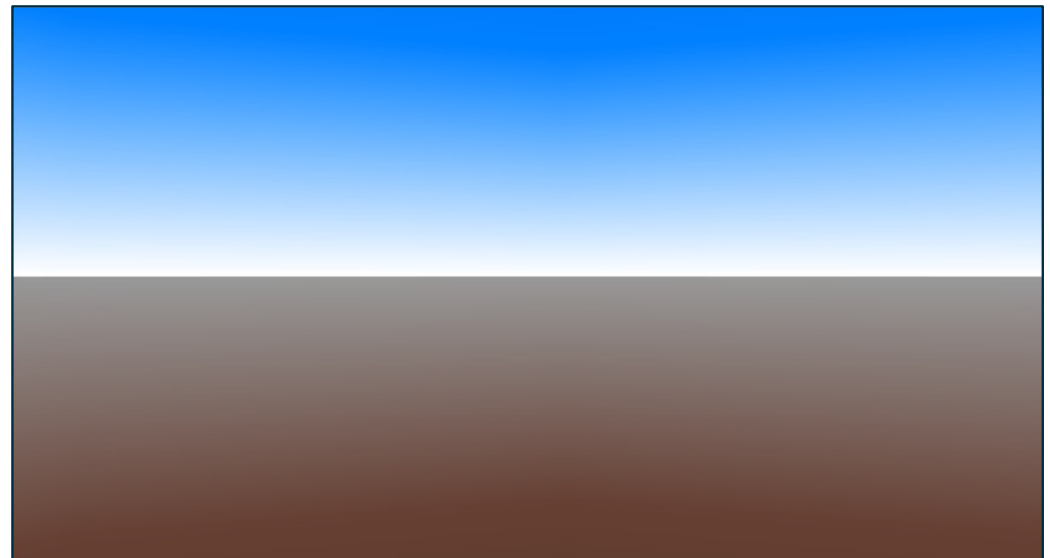
HORYZONT, PANORAMA I MGŁA – WĘZŁY BACKGROUND I FOG

#X3D V3.3 utf8

```
Background {  
  skyColor [  
    0.0 0.2 0.7,  
    0.0 0.5 1.0,  
    1.0 1.0 1.0  
  ]  
  skyAngle [ 1.309, 1.57]  
  
  groundColor [  
    0.1 0.10 0.0,  
    0.4 0.25 0.2,  
    0.6 0.60 0.6  
  ]  
  groundAngle [ 1.309,1.57]  
}
```



Źródło: [https://pl.wikipedia.org/wiki/Nadir_\(astronomia\)](https://pl.wikipedia.org/wiki/Nadir_(astronomia))



HORYZONT, PANORAMA I MGŁA – WĘZŁY BACKGROUND I FOG

Za pomocą węzła **Background** można utworzyć również panoramę otaczającą nasz świat. W skład takiej panoramy wchodzi **pliki graficzne** (JPG, PNG, GIF), które jako teksturę nakładamy na wielki sześcian otaczający cały nasz świat (sześcian ten jest domyślnie tworzony przez węzeł Background). Pola: **backUrl**, **bottomUrl**, **frontUrl**, **leftUrl**, **rightUrl**, **topUrl**.

HORYZONT, PANORAMA I MGŁA – WĘZŁY BACKGROUND I FOG

```
Fog {  
  exposedField SFColor  color          1 1 1  
  exposedField SFString fogType        "LINEAR"  
  exposedField SFFloat  visibilityRange 0  
  eventIn    SFBool  set_bind  
  eventOut   SFBool  isBound  
}
```

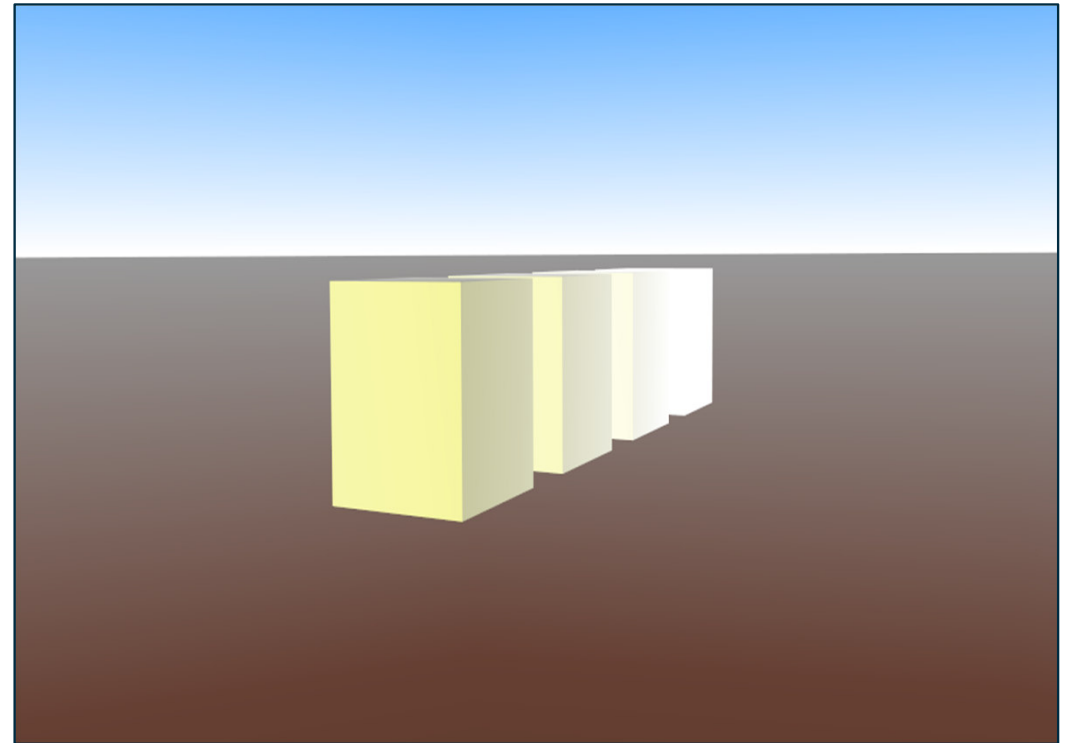
Pole **color** określa kolor mgły. Wartość **LINEAR** w polu **fogType** powoduje, że mgła rozpościera się **liniowo** we wszystkich kierunkach, natomiast gdy zostanie umieszczona tam wartość **EXPONENTIAL** uzyskamy efekt bardziej naturalny, zbliżony do rzeczywistości. W polu **visibilityRange** określa się najmniejszą odległość między użytkownikiem a obiektem, przy której obiekt zostaje zupełnie przez mgłę przesłonięty.

HORYZONT, PANORAMA I MGŁA – WĘZŁY BACKGROUND I FOG

#X3D V3.3 utf8

```
Background {  
  skyColor [  
    0.0 0.2 0.7,  
    0.0 0.5 1.0,  
    1.0 1.0 1.0  
  ]  
  skyAngle [ 1.309, 1.57]  
  
  groundColor [  
    0.1 0.10 0.0,  
    0.4 0.25 0.2,  
    0.6 0.60 0.6  
  ]  
  groundAngle [ 1.309,1.57]  
}  
  
Fog { color 1 1 1  
  visibilityRange 25  
  fogType "EXPONENTIAL" }  
  
DEF Kostka Shape {  
  appearance Appearance { material Material {  
    diffuseColor 1 1 0 }  
  }  
  geometry Box {}  
}
```

```
Transform {  
  translation 0 0 -3  
  children [USE Kostka]  
}  
  
Transform {  
  translation 0 0 -6  
  children [USE Kostka]  
}  
  
Transform {  
  translation 0 0 -9  
  children [USE Kostka]  
}
```



DZWIĘK I FILM – WĘZŁY SOUND, MOVIE TEXTURE I AUDIOCLIP

```
Sound {  
    exposedField SFVec3f  direction    0 0 1  
    exposedField SFFloat  intensity    1  
    exposedField SFVec3f  location     0 0 0  
    exposedField SFFloat  maxBack      10  
    exposedField SFFloat  maxFront     10  
    exposedField SFFloat  minBack      1  
    exposedField SFFloat  minFront     1  
    exposedField SFFloat  priority     0  
    exposedField SFNode   source       NULL  
    field      SFBool    spatialize    TRUE  
}
```

DZWIĘK I FILM – WĘZŁY SOUND, MOVIETEXTURE I AUDIOCLIP

- W polu **direction** określa się kierunek rozchodzenia się dźwięku. Pole **intensity** odpowiada za głośność z jaką dany dźwięk będzie odtwarzany (0 - cisza, 1 - pełna głośność).
- Pole **location** definiuje miejsce, z którego dany dźwięk będzie się rozchodził.
- Pole **spatialize** odpowiada za funkcję przestrzenności odgrywanego dźwięku. Jeżeli wartością tej funkcji będzie **FALSE** to dane dotyczące kierunku, w którym fale dźwiękowe mają się rozchodzić zostaną zignorowane.
- Pole **priority** stanowi odpowiedź dla przeglądarki dotyczącą dźwięku, który ma odtwarzać w momencie gdy na scenie zdefiniowanych jest więcej niż jedno źródło dźwięku.
- Pole **priority** podobnie jak pole **intensity** może przybierać wartości od 0 do 1, gdzie wartość 0 pomniejsza słyszalność danego dźwięku, a wartość 1 nadaje mu pierwszeństwo.

DZWIĘK I FILM – WĘZŁY SOUND, MOVIETEXTURE I AUDIOCLIP

Właściwości każdego dźwięku są takie, że jego fale rozchodzą się elipsoidalnie, w związku z czym dźwięk słychać również stojąc niedaleko od źródła dźwięku z tyłu. Zagadnienie to tłumaczy występowanie pól **minBack**, **maxBack** i **minFront**, **maxFront**. Wartość **minBack** określa odległość od źródła dźwięku "z tyłu", na której słyszalna jest pełna jego moc, a pole **maxBack** określa odległość również "z tyłu" źródła ale taką, na której dźwięk zupełnie zanika.

Analogicznie wygląda sytuacja z polami **minFront** i **maxFront** - odnosi się ona jedynie do odległości od źródła dźwięku zgodnej z kierunkiem transmisji. Im większy będzie odstęp między wartościami min a max, tym dźwięk będzie narastał łagodniej podczas zbliżania się do jego źródła.

DZWIĘK I FILM – WĘZŁY SOUND, MOVIE TEXTURE I AUDIOCLIP

W polu **source** umieszcza się węzeł **AudioClip** lub **MovieTexture**, w którym znajdzie się: ścieżka dostępu do pliku dźwiękowego oraz właściwości dotyczące trwania dźwięku w czasie. Podobnie jak węzeł **TimeSensor**, węzły **AudioClip** i **MovieTexture** są "węzłami zależnymi od czasu" czyli posiadają pola charakterystyczne dla tego typu węzłów i działają w podobny sposób.

DZWIĘK I FILM – WĘZŁY SOUND, MOVIE TEXTURE I AUDIOCLIP

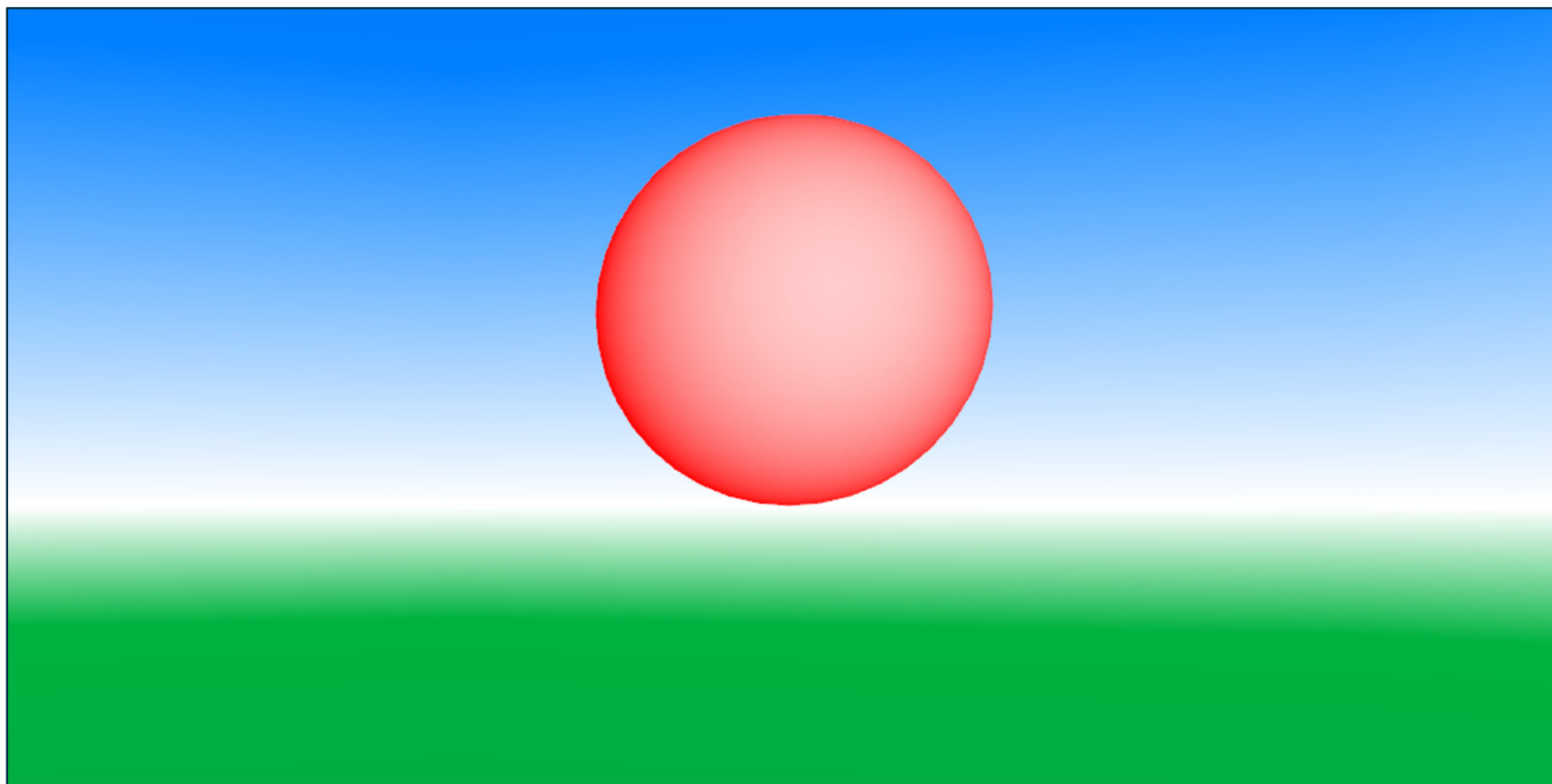
```
AudioClip {  
  exposedField SFString description      ""  
  exposedField SFBool   loop              FALSE  
  exposedField SFFloat  pitch             1.0  
  exposedField SFTime   startTime         0  
  exposedField SFTime   stopTime          0  
  exposedField MFString url               []  
  eventOut      SFTime   duration_changed  
  eventOut      SFBool   isActive  
}
```

DZWIĘK I FILM – WĘZŁY SOUND, MOVIE TEXTURE I AUDIOCLIP

Pola **loop**, **startTime** i **stopTime** – patrz węzeł **TimeSensor**. W polu **description** umieszcza się opis dźwięku (tylko niektóre przeglądarki obsługują tę funkcję). W polu **pitch** określa się czy dźwięk ma być odtwarzany z normalną prędkością 1 czy też ma być odtwarzany szybciej 1 > lub wolniej 1 <.

DZWIĘK I FILM – WĘZŁY SOUND, MOVIE TEXTURE I AUDIOCLIP

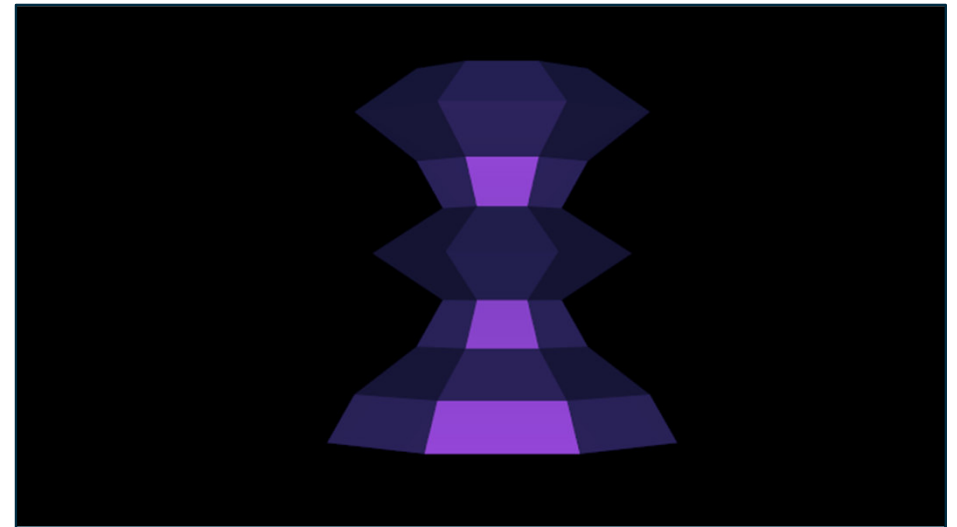
PRZYKŁAD: DZWIĘK NA KULI (viewpoint z przodu i na kuli).



TWORZENIE ZŁOŻONYCH OBIEKTÓW

Obiekty złożone tworzymy wykorzystując następujące węzły:

- **PointSet**,
- **IndexedLineSet**
- **IndexedFaceSet**,
- **Extrusion**.

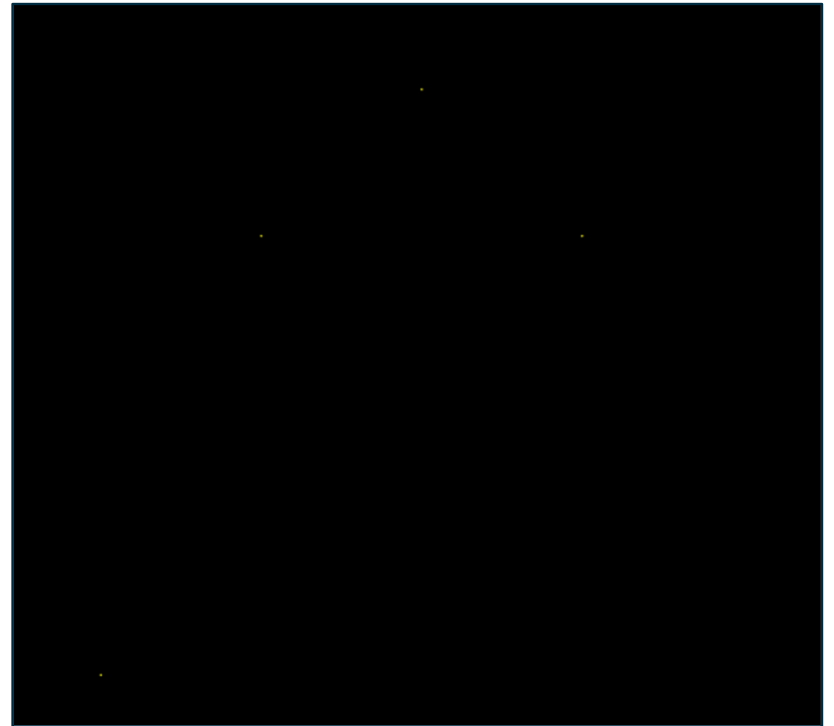


Tworzenie obiektów za pomocą powyższych węzłów polega na metodzie wyznaczania punktów w przestrzeni trójwymiarowej a następnie łączenia ich. Każdy z tych trzech węzłów w inny sposób traktuje określone punkty. Węzeł **IndexedFaceSet** używa ich do utworzenia **dwuwymiarowych wielokątów** - trzy połączone ze sobą punkty tworzą wielokąt. Z pojedynczych wielokątów dwuwymiarowych tworzone są potem obiekty trójwymiarowe. Węzeł **IndexedLineSet** tworzy **linie między minimum dwoma** punktami a węzeł **PointSet** tworzy **pojedyncze, świecące w przestrzeni punkty**. Węzeł **Extrusion** tworzy obiekty trójwymiarowe z **dwuwymiarowego planu** obiektu.

TWORZENIE ZŁOŻONYCH OBIEKTÓW

#X3D V3.3 utf8

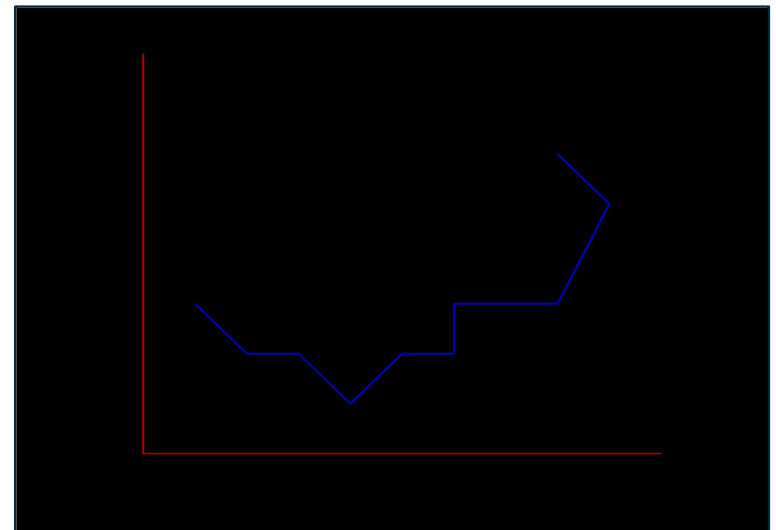
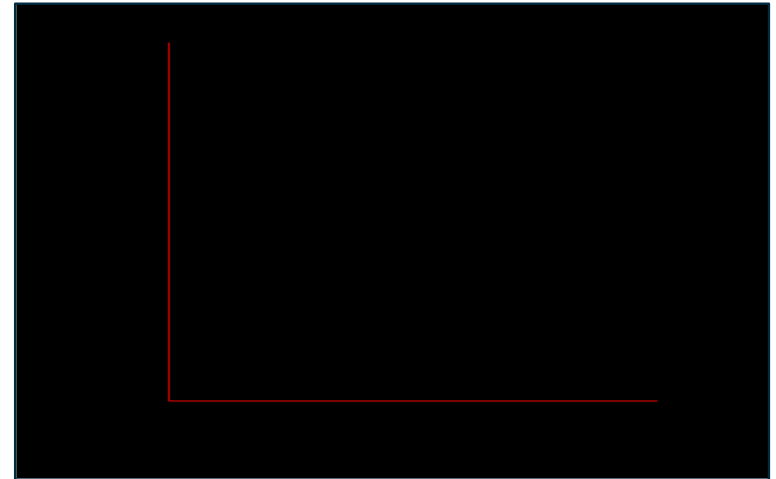
```
Shape {  
  appearance Appearance {  
    material Material {  
      diffuseColor 0.0 1.0 0.0  
      emissiveColor 1.0 1.0 0.0  
    }  
  }  
  geometry PointSet {  
    coord Coordinate {  
      point [  
        1.0 1.0 0.0, 2.0 4.0 0.0,  
        3.0 5.0 0.0, 4.0 4.0 0.0  
      ]  
    }  
  }  
}
```



TWORZENIE ZŁOŻONYCH OBIEKTÓW

#X3D V3.3 utf8

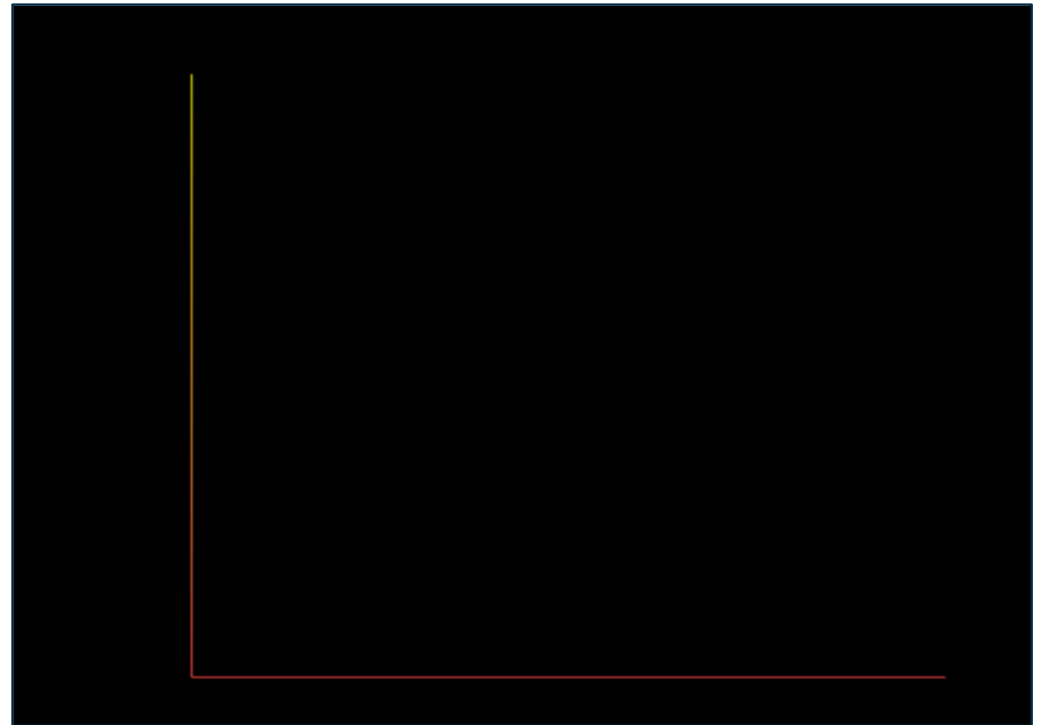
```
Shape {  
  appearance Appearance {  
    material Material {  
      emissiveColor 1.0 0.0 0.0  
    }  
  }  
  geometry IndexedLineSet {  
    coord Coordinate {  
      point [  
        0.0 8.0 0.0, 0.0 0.0 0.0, 10.0 0.0 0.0  
      ]  
    }  
    coordIndex [  
      0, 1, 2  
    ]  
  }  
}
```



TWORZENIE ZŁOŻONYCH OBIEKTÓW

#X3D V3.3 utf8

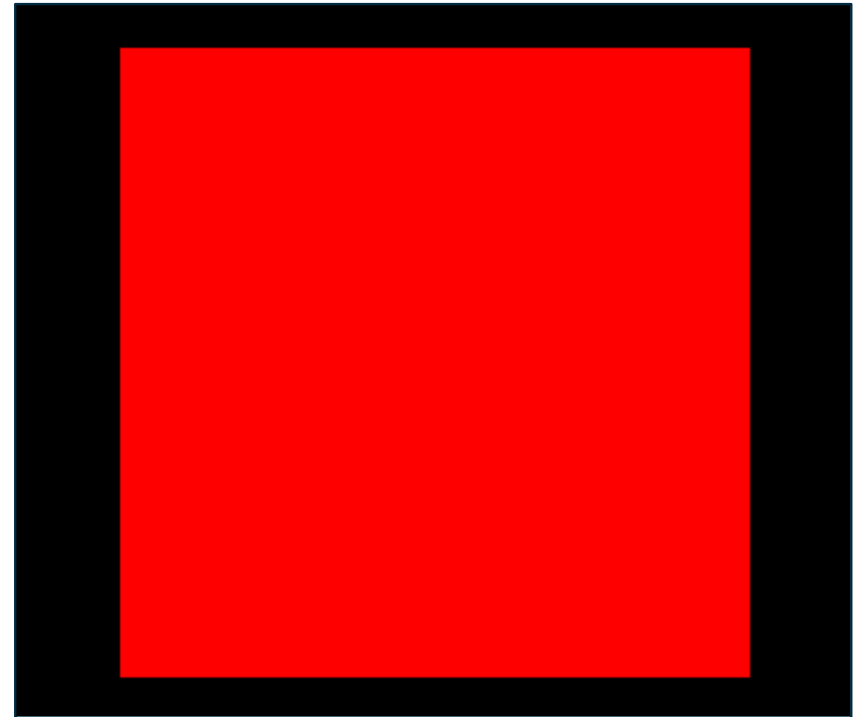
```
Shape {  
  geometry IndexedLineSet {  
    coord Coordinate {  
      point [  
        0.0 8.0 0.0, 0.0 0.0 0.0, 10.0 0.0 0.0  
      ]  
    }  
    coordIndex [  
      0, 1, 2  
    ]  
  }  
  colorPerVertex TRUE  
  color Color {  
    color [  
      1.0 1.0 0.0,  
      1.0 0.3 0.3,  
      1.0 0.3 0.3  
    ]  
  }  
}
```



TWORZENIE ZŁOŻONYCH OBIEKTÓW

#X3D V3.3 utf8

```
Shape {  
  appearance Appearance {  
    material Material {  
      diffuseColor 1.0 0.0 0.0  
    }  
  }  
  geometry IndexedFaceSet {  
    coord Coordinate {  
      point [  
        0.0 0.0 0.0,  
        1.0 0.0 0.0,  
        1.0 1.0 0.0,  
        0.0 1.0 0.0  
      ]  
    }  
    coordIndex [  
      0, 1, 2, 3  
    ]  
  }  
}
```

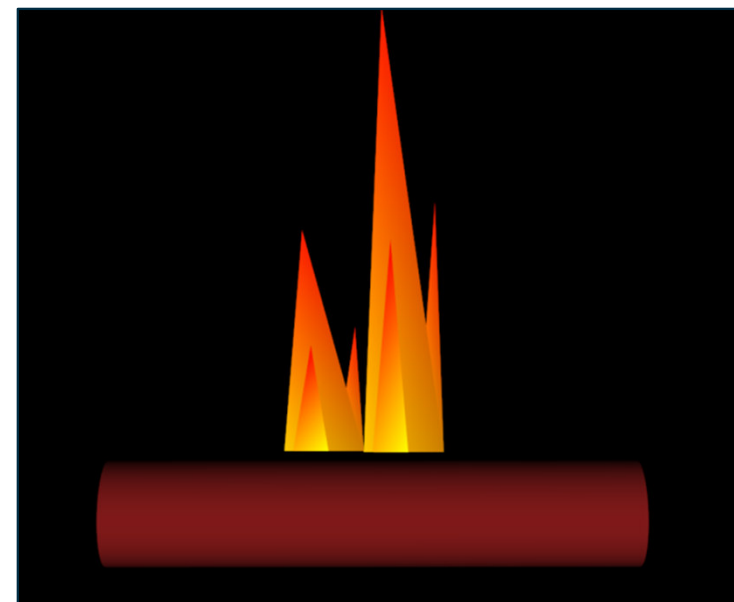


TWORZENIE ZŁOŻONYCH OBIEKTÓW

#X3D V3.3 utf8

```
Transform {  
  translation 0.0 -0.7 0.0  
  rotation 0.0 0.0 1.0 -1.57  
  children Shape {  
    appearance Appearance {  
      material Material {  
        diffuseColor 0.5 0.1 0.1  
      }  
    }  
    geometry Cylinder {  
      height 6.0  
      radius 0.6  
    }  
  }  
}  
  
DEF Plomien Shape {  
  geometry IndexedFaceSet {  
    coord Coordinate {  
      point [  
        -0.1 0.0 0.0, -0.8 2.5 0.0, -1.0 0.0 0.0,  
        -0.5 0.0 0.01, -0.7 1.2 0.01, -0.9 0.0 0.01,  
        -0.1 0.0 0.0, -0.2 1.4 0.0, -0.4 0.0 0.0  
      ]  
    }  
  }  
}
```

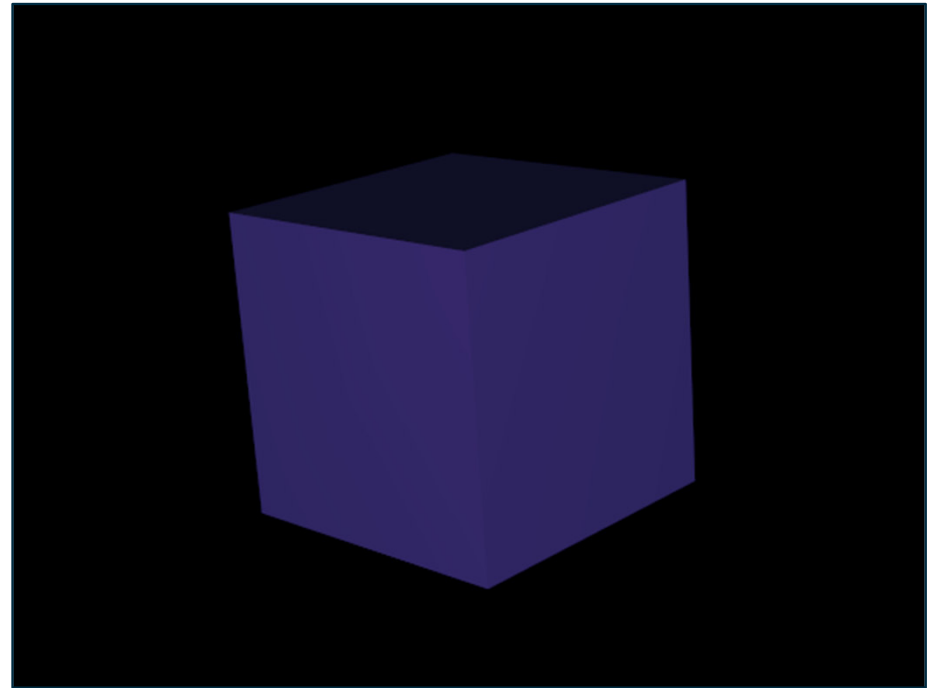
```
coordIndex [  
  0, 1, 2, -1, 3, 4, 5, -1,  
  6, 7, 8, -1  
]  
colorPerVertex TRUE  
color Color {  
  color [  
    1.0 1.0 0.0, 1.0 0.0 0.0, 1.0 0.7 0.0,  
    0.8 0.5 0.0, 1.0 0.1 0.0, 1.0 0.8 0.0  
  ]  
}  
colorIndex [  
  3, 4, 5, 0, 0, 1, 2, 0,  
  3, 4, 5, 0, 0, 1, 2, 0  
]  
}  
  
Transform {  
  translation 0.9 0.0 0.1  
  scale 1.0 2 1.0  
  children USE Plomien  
}
```



TWORZENIE ZŁOŻONYCH OBIEKTÓW

#X3D V3.3 utf8

```
Shape {  
  appearance Appearance { material Material {  
    ambientIntensity 0.3  
    diffuseColor 0.2 0.2 0.5  
    specularColor 0.8 0.2 0.8  
    shininess 0.05  
  }  
  geometry Extrusion {  
    crossSection [1 1, 1 -1, -1 -1,  
                 -1 1, 1 1]  
    spine [ 0 0 0, 0 2 0]  
  }  
}
```



RZEŻBA TERENU – WĘZEŁ ELEVATIONGRID

```
ElevationGrid {  
    eventIn      MFFloat set_height  
    exposedField SFNode  color      NULL  
    exposedField SFNode  normal     NULL  
    exposedField SFNode  texCoord   NULL  
    field        MFFloat height     []  
    field        SFBool  ccw         TRUE  
    field        SFBool  colorPerVertex TRUE  
    field        SFFloat creaseAngle 0  
    field        SFBool  normalPerVertex TRUE  
    field        SFBool  solid        TRUE  
    field        SFInt32 xDimension  0  
    field        SFFloat xSpacing    1.0  
    field        SFInt32 zDimension  0  
    field        SFFloat zSpacing    1.0  
}
```


RZEŹBA TERENU – WĘZEŁ ELEVATION GRID

METODYKA MANUALNEGO PROJEKTOWANIA:

- Pierwszym krokiem jest budowa dwuwymiarowej siatki rozciągniętej pomiędzy osią X i Z a następnie ustalenie wysokości punktów w których siatka się przecina. Siatkę tworzy się za pomocą pól **xDimension** i **zDimension** gdzie definiuje się w nich ilość linii przecinających się. W polach **xSpacing** i **zSpacing** ustala się odległość między tymi liniami (rozrzedzenie lub zagęszczenie siatki).
- Kolejnym krokiem jest definicja pola **height**, gdzie każdemu punktowi przecięcia linii siatki podporządkowuje się wysokość wierzchołków. Pierwsza wartość w tym polu odnosi się do punktu (0, 0) w układzie (X, Z), następne wartości odnoszą się do kolejnych punktów znajdujących się na osi X (wartość określona w polu **xDimension**). Kiedy wszystkie wartości na osi X dla Z=0 zostaną opisane wtedy kolejnym punktem, który należy opisać w polu **height** będzie pierwszy punkt znajdujący się na dodatniej stronie osi Z dla X=0. Kolejno opisuje się pozostałe punkty od X=0 do X=n, aż do chwili opisanie wszystkich punktów przecięcia siatki. Ilość wartości w polu **height** jest równa działaniu **xDimension * zDimension**.

RZEŻBA TERENU – WĘZEŁ ELEVATION GRID

- W polu **ccw** określa się porządek współrzędnych wierzchołków użytych w celu utworzenia bryły. Jeżeli w polu tym znajdzie się wartość **TRUE** to oznacza, że wierzchołki tworzące jedną ścianę bryły definiowane są w kierunku przeciwnym do ruchu wskazówek zegara. Jeżeli w polu tym znajdzie się wartość **FALSE** to oznacza, że wierzchołki będą definiowane zgodnie z ruchem wskazówek zegara. W polu **solid** wartość **FALSE** oznacza, że każdy z wielokątów wchodzących w skład definiowanej bryły będzie renderowany niezależnie od tego, czy użytkownik ma ku niemu zwrócony wzrok czy też nie.
- Pole **color** pozwala przypisać inny kolor każdemu wierzchołkowi bądź każdemu wielokątowi zdefiniowanemu w węźle **ElevationGrid**. W polu **color** można zdefiniować węzeł **Color**, jednak wartości które mogą się w nim znaleźć, zależne są od pola **colorPerVertex**. Pole **colorPerVertex** określa czy kolory zdefiniowane w polu **color** będą przypisane do każdego wierzchołka (**TRUE**) czy do każdego czworoboku (**FALSE**).

RZEŻBA TERENU – WĘZEŁ ELEVATION GRID

▪ Pole **normal** przypisuje każdemu wierzchołkowi bądź czworobokowi **wektory normalne**. W polu **normal** można zdefiniować węzeł **Normal**, jeżeli nie zostanie to zrobione przeglądarka automatycznie wygeneruje wektory normalne korzystając z wartości pola **creaseAngle**. Jeśli kąt między dwoma płaszczyznami będzie mniejszy od kąta zawartego w polu **creaseAngle** to krawędzie tych dwóch płaszczyzn powinny być delikatnie wycieniowane. Podobnie jak w przypadku kolorów, cieniowanie można przypisać albo do wierzchołków albo do wielokątów (pole **normalPerVertex**).

W polu **texCoord** można zdefiniować węzeł **TextureCoordinate** odpowiadający za nałożenie tekstury na obiekty o budowie wierzchołkowej.

RZEŹBA TERENU – WĘZEŁ ELEVATION GRID

#X3D V3.3 utf8

```
Shape {
  appearance Appearance {
    material Material { }
  }
  geometry ElevationGrid {
    xDimension 9
    zDimension 9
    xSpacing 1.0
    zSpacing 1.0
    creaseAngle 1.57
    height [
      0.0, 0.0, 0.5, 1.0, 0.5, 0.0, 0.0, 0.0, 0.0,
      0.0, 0.0, 0.0, 0.0, 2.5, 0.5, 0.0, 0.0, 0.0,
      0.0, 0.0, 0.5, 0.5, 3.0, 1.0, 0.5, 0.0, 1.0,
      0.0, 0.0, 0.5, 2.0, 4.5, 2.5, 1.0, 1.5, 0.5,
      1.0, 2.5, 3.0, 4.5, 1.5, 3.5, 3.0, 1.0, 0.0,
      0.5, 2.0, 2.0, 2.5, 3.5, 4.0, 2.0, 0.5, 0.0,
      0.0, 0.0, 0.5, 1.5, 1.0, 2.0, 8.0, 1.5, 0.0,
      0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 4.0, 1.5, 0.5,
      0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.5, 0.0, 0.0,
    ]
  }
  colorPerVertex TRUE
  color Color {
    color [
      1.0 0.4 1.0, 0.0 0.3 1.0, 1.0 0.5 0.1,
      0.2 0.6 0.0, 0.0 0.5 0.1, 1.0 0.3 1.0,
      0.0 0.4 1.0, 0.0 0.3 1.0, 1.0 0.3 1.0,
    ]
  }
}
```

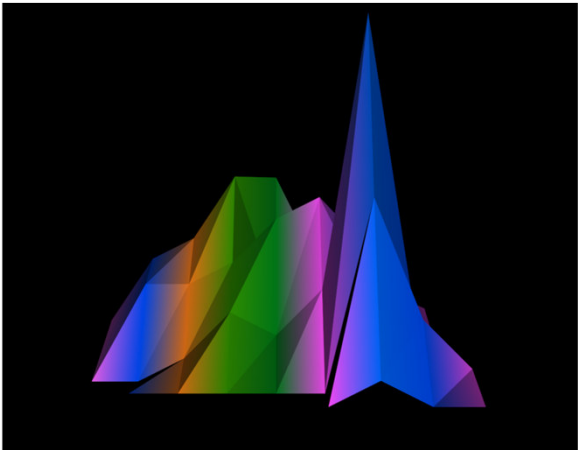
```
1.0 0.4 1.0, 0.0 0.3 1.0, 1.0 0.5 0.1,
0.2 0.6 0.0, 0.0 0.5 0.1, 1.0 0.3 1.0,
0.0 0.4 1.0, 0.0 0.3 1.0, 1.0 0.3 1.0,

1.0 0.4 1.0, 0.0 0.3 1.0, 1.0 0.5 0.1,
0.2 0.6 0.0, 0.0 0.5 0.1, 1.0 0.3 1.0,
0.0 0.4 1.0, 0.0 0.3 1.0, 1.0 0.3 1.0,

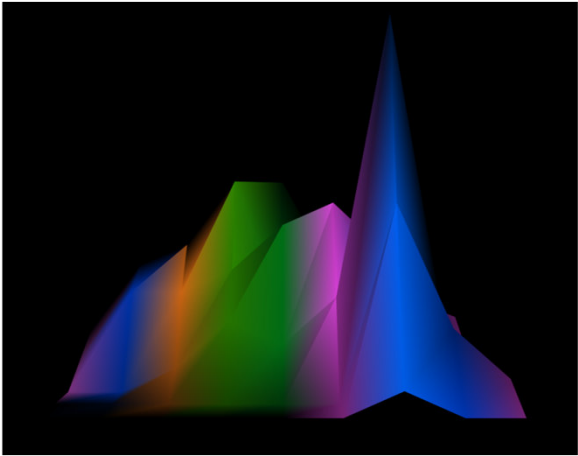
1.0 0.4 1.0, 0.0 0.3 1.0, 1.0 0.5 0.1,
0.2 0.6 0.0, 0.0 0.5 0.1, 1.0 0.3 1.0,
0.0 0.4 1.0, 0.0 0.3 1.0, 1.0 0.3 1.0,

1.0 0.4 1.0, 0.0 0.3 1.0, 1.0 0.5 0.1,
0.2 0.6 0.0, 0.0 0.5 0.1, 1.0 0.3 1.0,
0.0 0.4 1.0, 0.0 0.3 1.0, 1.0 0.3 1.0,

1.0 0.4 1.0, 0.0 0.3 1.0, 1.0 0.5 0.1,
0.2 0.6 0.0, 0.0 0.5 0.1, 1.0 0.3 1.0,
0.0 0.4 1.0, 0.0 0.3 1.0, 1.0 0.3 1.0,
```



creaseAngle 0.0

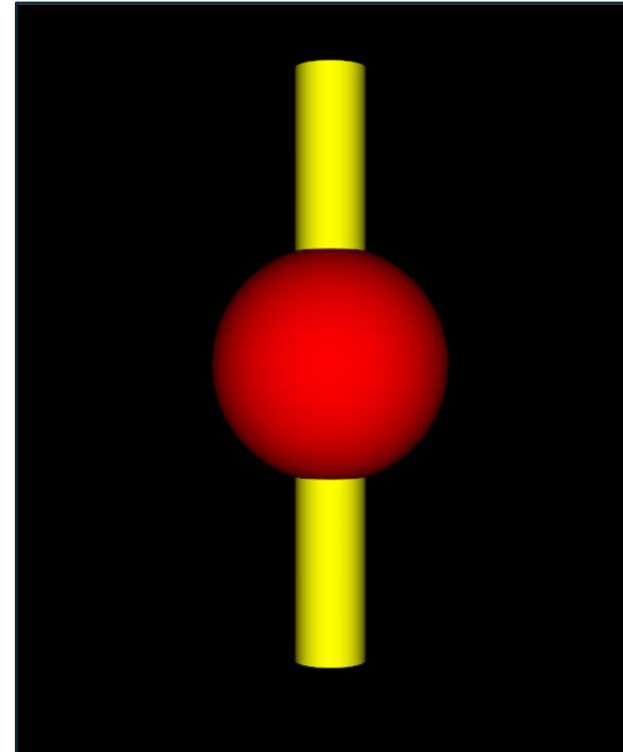


creaseAngle 1.57

WĘZEŁ GROUP

#X3D V3.3 utf8

```
DEF mojagrupa Group {  
  children [  
    Shape {  
      appearance Appearance {  
        material DEF Kula Material {  
          diffuseColor 1.0 0.0 0.0  
        }  
      }  
      geometry Sphere { }  
    },  
  
    Shape {  
      appearance Appearance {  
        material Material {  
          diffuseColor 1 1.0 0  
        }  
      }  
      geometry Cylinder {  
        radius 0.3  
        height 5.0  
      }  
    }  
  ]  
}
```



WĘZEŁ WHICH

#X3D V3.3 utf8

Switch {

 whichChoice 3

 choice [

```
    Group {
      children [
        DEF Akronim Shape {
          appearance Appearance {
            material Material {
              emissiveColor 1.0 1.0 1.0
              diffuseColor 0.0 0.0 0.0
            }
          }
        }
        geometry Text {
          string "AGH - 0"
          fontStyle FontStyle {
            justify "MIDDLE"
          }
        }
      ],
    },
  ]
},
```

```
Group {
  children [
    DEF Akronim Shape {
      appearance Appearance {
        material Material {
          emissiveColor 1.0 1.0 1.0
          diffuseColor 0.0 0.0 0.0
        }
      }
      geometry Text {
        string "AGH - 1"
        fontStyle FontStyle {
          justify "MIDDLE"
        }
      }
    },
  ],
},
Group {
  children [
    DEF Akronim Shape {
      appearance Appearance {
        material Material {
          emissiveColor 1.0 1.0 1.0
          diffuseColor 0.0 0.0 0.0
        }
      }
      geometry Text {
        string "AGH - 2"
        fontStyle FontStyle {
          justify "MIDDLE"
        }
      }
    },
  ],
},
```

```
    },
  ],
Group {
  children [
    DEF Akronim Shape {
      appearance Appearance {
        material Material {
          emissiveColor 1.0 1.0 1.0
          diffuseColor 0.0 0.0 0.0
        }
      }
      geometry Text {
        string "AGH - 3"
        fontStyle FontStyle {
          justify "MIDDLE"
        }
      }
    },
  ],
},
  ],
}
```

AGH - 3

PROTO

#X3D V3.3 utf8

PROTO MojKolor [] {

```
  Material {  
    diffuseColor 1.0 1.0 0.0  
  }
```

```
}
```

Shape {

```
  appearance Appearance {  
    material MojKolor { }
```

```
  }  
  geometry Box { size 10.0 2.0 0.1 }
```

```
}
```



PROTO

#X3D V3.3 utf8

EXTERNPROTO Miedz [] "mojaBiblioteka.x3dv#Miedz"

```
Shape {  
  appearance Appearance {  
    material Miedz { }  
  }  
  geometry Box { size 10.0 3.0 0.5 }  
}
```



#X3D V3.3 utf8

```
PROTO Zloto [ ] {  
  Material {  
    ambientIntensity 0.4  
    diffuseColor 0.22 0.15 0.0  
    specularColor 0.71 0.70 0.56  
    shininess 0.16  
  }  
}  
PROTO Aluminium [ ] {  
  Material {  
    ambientIntensity 0.3  
    diffuseColor 0.30 0.30 0.50  
    specularColor 0.70 0.70 0.80  
    shininess 0.10  
  }  
}  
PROTO Miedz [ ] {  
  Material {  
    ambientIntensity 0.26  
    diffuseColor 0.30 0.11 0.00  
    specularColor 0.75 0.33 0.00  
    shininess 0.08  
  }  
}
```

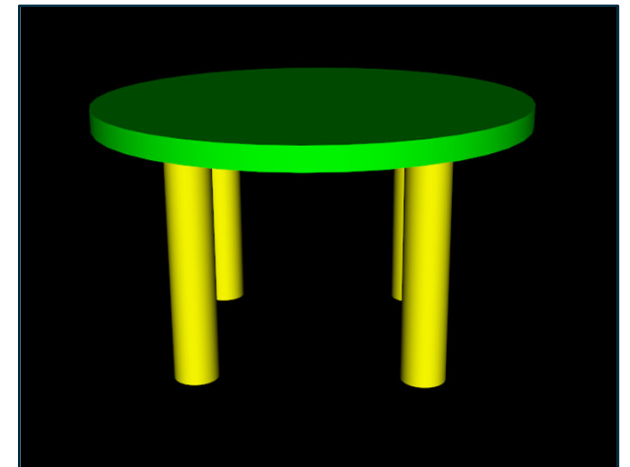

PROTO

#X3D V3.3 utf8

```
PROTO MojPrototyp
[ field SFCOLOR kolorNogi 1.0 0.0 0.0
  field SFCOLOR kolorSiedzenie 0.0 1.0 0.0 ]
{
  Transform {
    children [
      Transform {
        translation 0 0.6 0
        children
        Shape {
          appearance Appearance {
            material Material { diffuseColor IS kolorSiedzenie }
          }
          geometry Cylinder { height 0.1 radius 1.0 }
        }
      ]
    }

  Transform {
    translation -0.5 0 -0.5
    children
    DEF Noga Shape {
      appearance Appearance {
        material Material { diffuseColor IS kolorNogi }
      }
      geometry Cylinder { height 1 radius 0.1 }
    }
  }
}
```

```
Transform {
  translation 0.5 0 -0.5
  children USE Noga
}
Transform {
  translation -0.5 0 0.5
  children USE Noga
}
Transform {
  translation 0.5 0 0.5
  children USE Noga
}
}
}
}
MojPrototyp {
  kolorNogi 1 1 0
  kolorSiedzenie 0 1 0
}
```



ĆWICZENIA

UWAGI DO ĆWICZEŃ:

1. Zadania wykonujemy samodzielnie ☺.
2. Każde zadanie modelowe zapisujemy do osobnego pliku nadając mu nazwę odpowiadającą numerowi zadania.
3. Oficjalne specyfikacje dla standardu X3D (kodowanie klasyczne):

<https://www.web3d.org/documents/specifications/14772/V2.0/>

Pliki z swoimi rozwiązaniami umieszczamy w swoich indywidualnych katalogach na MS Teams.

ĆWICZENIA

01

Wykorzystując opracowaną na wcześniejszych zajęciach scenę 3D **LABIRYNT** uzupełnić kod programując:

- a) przycisk otwierający drzwi do labiryntu (**węzeł TouchSensor**),
- b) pochodnię imitującą płomień (zmiana koloru czerwony-żółty-czerwony, **węzeł ColorInterpolator**),
- c) ukryta na ścianie informacja/wskazówka która będzie widoczna po włączeniu światła (**węzeł PointLight**),
- d) zaprojektować obszar aktywny na wkroczenie użytkownika, aktywujący animację poruszającego się i znikającego duszka (**węzeł ProximitySensor**),
- e) wykorzystaj polecenie **PROTO** do definicji prototypu np. stół/krzesło/obraz/inny obiekt według uznania,
- f) stworzyć własną rzeźbę terenu (**węzeł ElevationGrid**),
- g) mgła (**węzeł Fog**),
- h) zaprojektować panoramę dla labiryntu (**węzeł Background**),
- i) portal do innego świata (**węzeł Collision**),
- j) sekretne przejście (**węzeł PlaneSensor** lub **węzeł CylinderSensor**).

**DZIĘKUJĘ
ZA UWAGĘ**

LITERATURA

1. <https://www.web3d.org>
2. <https://www.x3dom.org>
3. <https://x3dgraphics.com>
4. K. Dąbkowski, VRML 97 trzeci wymiar sieci, Wydawnictwo MIKOM, Warszawa, 1998
5. S. Bucaro, X3Dom Primer: Learn the basics of how to code 3D objects and scenes to display on the web, bucarotechelp.com , 2018
6. D. Brutzman , L. Daly, X3D: Extensible 3D Graphics for Web Authors, Morgan Kaufmann, 2017